

Vision-Based Reinforcement Learning for Autonomous Drone Racing

Robert Oxley

Abstract—Autonomous drone racing requires an agent to fly through a sequence of gates within course boundaries using exteroceptive and proprioceptive sensors onboard. This work presents an autonomy stack developed for the Anduril AI Grand Prix, structured around a clear progression of goals: first achieve reliable flight using privileged ground-truth state, then close the gap to pure onboard sensing (monocular perception + IMU), then demonstrate transfer across simulators by completing Virtual Qualifier 1, and finally push performance on Virtual Qualifier 2 while preparing the policy for eventual sim-to-real transfer. I present a PPO policy that is trained on privileged gate states inside a modified Flightmare environment using a three-phase curriculum and domain randomization. The same policy is deployed with a perception pipeline consisting of a fine-tuned YOLO pose estimator, Perspective-n-Point, a Kalman filter, and uncertainty-gated association to output body rates and continuous thrust at 120 Hz for an underactuated quadrotor. In controlled evaluation on noisy monocular perception, the vision-based system retains approximately 65% of the privileged-state completion rate. The full autonomy stack completed Virtual Qualifier 1 and progressed partway through Virtual Qualifier 2. The result is a practical demonstration of moving from privileged simulation training to onboard-only inference while deliberately preparing the policy for later real-world transfer.

I. OVERVIEW

After developing an interest in generating trajectories for autonomous robotics, I entered the Anduril AI Grand Prix to gain practical experience applying reinforcement learning to autonomous quadrotor flight in both simulated and more realistic conditions. The competition required flying through a sequence of gates using only onboard sensors: a forward-facing camera and an Inertial Measurement Unit (IMU), with no external localization allowed.

Approaches to the task of autonomous navigation through gates, including MonoRace [1] and Swift [8] inspired my design for the policy built. To develop a robust policy that meets the competition requirements of only onboard sensors during inference, I modified a Flightmare simulator [2], a C++ and Python physics simulation and rendering engine for autonomous drones. This setup allowed me to structure the work as a deliberate progress of my goals: First, create an autonomy stack (Fig. 1) that can reliably navigate gates using privileged ground-truth state. Next, close the distribution gap so the same policy can operate on pure onboard sensing using vision and IMU estimates. Then, demonstrate that the policy can transfer between simulators by completing Virtual Qualifier 1 of the Anduril AI Grand Prix, which is hosted in a separate simulator made by the Drone Champions League (DCL). Finally, push performance on Virtual Qualifier 2

using more complex simulated conditions and domain randomization, leaving the policy better prepared for eventual sim-to-real transfer. This paper describes the autonomy stack developed to meet these goals and evaluates its performance on both privileged and vision-based inference.

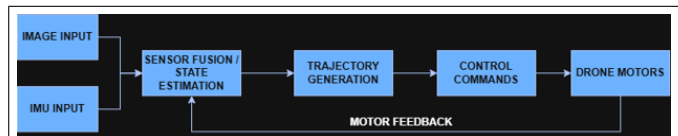


Fig. 1. Conceptual Control Architecture for an autonomous drone given the input of an IMU and Camera.

II. APPROACH

I address the goals with a two-stage approach. A Proximal Policy Optimization (PPO) algorithm is trained on a 34-dimensional state and outputs collective thrust and body-rate commands for an underactuated quadrotor at 120 Hz. The reward prioritizes reliable gate clearance over lap time. Training is performed in a modified Flightmare simulator across a three-phase curriculum of increasing complexity. The drone is trained on privileged ground-truth gate state and then deployed using monocular vision estimates. This enables direct evaluation of both control performance under complex conditions and the distribution gap between privileged and onboard sensing.

A. Environment

The Anduril AI Grand Prix provided a simulator that had a rigid-body drone flight model that included thrust generation, aerodynamic drag, gravity, and collision physics with an update rate of 120 Hz. The gates had a 1.5x1.5 meter inner opening and were clear and easily identifiable. Virtual Qualifier 1 contained fewer than 10 gates with minimal distractions and clear visibility, while Virtual Qualifier 2 had fewer than 20 gates, lighting changes, 3D objects, and obstacles. State estimation had to rely on onboard sensing, which included a forward-facing first-person camera and an IMU. Training and evaluation of the policy run in a modified version of Flightmare. The most notable changes in Flightmare were made to prevent “reward hacking” from the PPO policy. These changes include:

- Two-sided gate collision detection to prevent the policy from crossing a gate and then colliding with the object. In training, the negative reward of colliding with a gate is

$$-0.5 \cdot n_{hits} \quad (1)$$

*This work was not supported by any organization

This is because stopping the episode due to a collision would give PPO, an on-policy algorithm which does not store experience from previous iterations [3], fewer data to step towards the positive gradient.

- Dynamic lateral/vertical walls to prevent the policy from generating a trajectory that uses wide or high/low routes towards the gate. The lowest possible vertical floor is generated by

$$\min(z_{lowest} - (2.0 + 2.0 \cdot phase), -1.0) \quad (2)$$

The lowest possible lateral wall is generated by

$$\min(x_{lowest}, 0) - \delta \sim U(3.0, 6.0) \quad (3)$$

The maximum is used for the right side. This helps in the event (such as VQ2 in the Anduril AI Grand Prix) where the course is inside or with obstacles surrounding the gates, without losing speed due to obstacle avoidance.

- Domain randomization that can be activated through a flag (in this case, during phase 3 of the training process). This updates the drone’s dynamics every episode reset, following the approach of [4], which samples each parameter independently, including mass, thrust, motor time constant, kappa. This helped to improve simulation-to-real transfer and is inferred to be the primary reason the policy could interact with both the Flightmare simulator and DCL due to the large distribution of drone dynamics that was supported, though no direct test was done to prove this.

The monocular RGB camera is also embedded into the drone, a +20 pitch angle, z axis offset of 0.3 meters, vertical FOV of 90, and outputs images that are 640x360. The environment also includes a flag to configure the initial random rotation of the drone in the starting state or to launch from the ground. Finally, the entire simulator was updated to Python 3.9, Ubuntu 20.04, and CUDA 13.1 for newer Docker support and Stable-Baselines3 / PyTorch support.

B. Model

The policy uses the Stable-Baselines3 PPO algorithm, which is an actor-critic reinforcement learning algorithm designed for stable and efficient policy updates. A stable RL algorithm is critical for a quadrotor, as they are inherently unstable systems that require an impossible model of each aspect of the drone’s dynamics [5]. The hidden layers of the policy and value function each use a 64x64 MLP, while the input is a 34-dimensional observation state and the output is 4 continuous control mappings. The observations are defined as follows

$$x = [x, y, z, q_R, v_x, v_y, v_z, \omega_p, \omega_q, \omega_r, u_{prev}, (g_x, g_y, g_z)^4] \quad (4)$$

$$u = [t, \phi, \theta, \psi] \in [-1, 1] \quad (5)$$

Here, (x, y, z) equals the position of the gate estimate relative to the drone-body. Q_R is the 9-dimensional rotation matrix that was chosen over quaternions since it is a

continuous embedding of SO(3) rather than a discontinuity that degrades the network approximation due to the double cover of a quaternion [6]. The observations (v_x, v_y, v_z) and $(\omega_\theta, \omega_\phi, \omega_\psi)$ are the linear and angular velocities of the quadrotor, u_{prev} is the previous action. $(g_x, g_y, g_z)^4$ Are the 4 (x, y, z) coordinates of each corner of the gate estimate that allow the policy to thread a gate without hitting the frame. Finally, the output vector $u = [t, \phi, \theta, \psi]$ represents the continuous thrust, roll, pitch, and yaw body rates of the quadrotor. Each observation is normalized by the running mean/std in the state, but the angular velocities are excluded and then normalized by ω_{max} instead so that it can handle different magnitudes of velocities without being out of distribution (OOD)

C. Training

Training a policy on autonomously traversing a series of gates is done by running 3 phases of training, which are 1.2×10^8 timesteps for phases 1 and 2, and 3.2×10^8 for phase 3. They are warm started from each phase’s trained weights/normalizer, and are as follows:

- Phase 1: Forward-facing gates, with small lateral/vertical shifts and a time limit of 20 seconds.
- Phase 2: Rotated gates with longer lateral/vertical shifts and a time limit of 20 seconds, deviations from the camera result in a dead-reckoned gate estimate to match realistic expectations and only updated every 30 Hz to not rely on information each timestep.
- Phase 3: Sharper gate rotations with steeper lateral/vertical shifts. Deviating from the forward vector of the current gate is penalized more severely (-0.1 instead of -0.01), and the gate crossing bonus increases from 10 to 30. Structured noise is added to the gate estimates for robustness towards estimates that may be slightly wrong. In addition, Domain Randomization is introduced.

Each phase has a constant chance of encountering a gate at the same vertical level as the previous, which in order are [80%, 60%, 40%]. This, along with the vertical and lateral shifts, covers the edge cases observed for a gate within the scope of the task and the specifications given. Each phase uses curriculum learning to increase the number of gates during training, from 1 – > 14. This increases when success clears a threshold formula of $\min(0.65, 0.85^n)$, where n is the amount of gates and 0.85 represents the per-gate reliability that has to be met, the maximum is capped at 65% success due to the risk of overfitting a policy for early gates. This represents the horizon length per task difficulty increase, allowing the model to receive positive rewards after threading n gates before increasing the length. Entropy is handled through a reverse entropy schedule. It starts from 0.005 and decay as the gated curriculum advances, towards 0.001.

D. Reward Structure

In 2025, the ‘MonoRace’ approach [1] won the 2025 A2RL Drone Championship hosted by A2RL and DCL. This

system utilized a single rolling-shutter CMOS camera and an Inertial Measurement Unit (IMU) to operate at 500 Hz to emphasize a high-speed, agile flight autonomy stack. The approach I present adopts a similar reward structure. The reward function consists of a dense per-step shaping sum that is applied every timestep. Table 1 showcases this reward, the time-weighted completion rate, timeout, gate reward, and frame hit reward are each terms that are given when the drone completes the event required. Progress, angular velocity, action, and perception are dense terms given every timestep.

TABLE I
REWARD STRUCTURE COEFFICIENTS

Coefficient	Phase 2	Phase 3
α_p	1	-
α_ω	-0.001	-
α_α	-0.01	-
α_c	-0.01	-0.1
α_g	+10	+30
α_f	$50 + 25 \cdot (1 - \frac{steps}{n_{steps}})$	-
α_o	-10	-
α_h	-0.5	-

The dense reward is found in Equation 6. Equations 7, 8, and 9 define the distance covered by the quadrotor per timestep, 8 prevents instantaneous switching of motors, known as a "bang-bang" motion, and 9 is the perception penalty when the quadrotor deviates at least 60 degrees from the current gate pose. $\hat{g}$ represents the unit vector from the drone to the current gate center, while $\hat{c}$ is the camera's forward axis and d is the distance between the quadrotor and the current gate.

$$r_t^{step} = \alpha_p \Delta d_t + \alpha_\omega \|\omega_t\|^2 + \alpha_a \sum_j \epsilon_{t,j} + \alpha_c c_t \quad (6)$$

$$\Delta d_t = clip(d_{t-1} - d_t, -v_{max} \Delta t, v_{max} \Delta t) \quad (7)$$

$$\epsilon_{t,j} = max(0, |a_{t,j} - a_{t-1,j}| - 0.5) \quad (8)$$

$$c_t = max(0, \cos 60^\circ - \langle \hat{g}_t, \hat{c}_t \rangle) \cdot 1_{d_t \geq 2} \quad (9)$$

The rewards given upon an event being observed are found in Equation 10. Equation 11 represents the total reward equation¹.

$$r_t^{event} = 1_{thread} \cdot \alpha_g - 0.5 n_t^{hit} + 1_{finish} \cdot \alpha_f \quad (10)$$

$$r_t = 1_{terminal} \cdot \alpha_o + (1 - 1_{terminal})(r_t^{step} + r_t^{event}) \quad (11)$$

¹Boundary and pad-launch violations incur α_o twice

E. Perception

During training the model received privileged ground-truth gate estimates; however during inference where the stack is restricted to onboard sensors there must exist a pipeline to convert the monocular camera image into the gate estimates. This autonomy stack utilizes a fine-tuned YOLO pose estimation model to give the 4 inner corner key points of a detected gate in the format tl, tr, br, bl . A confidence threshold of 70% is applied, as a false positive applied to a complex system can be catastrophic, therefore for precision the vision model must emit confidence values above it. Given the known geometry of the gate (1.5x1.5 meter inner width/height) and the intrinsic, extrinsic characteristics of the camera, IPPE-Square can be used during Perspective-n-Point (PnP) computation to avoid the need for a learned depth net or any scale ambiguity [7]. When a detection is not found, the previous detection is dead reckoned until a new detection is found. A Kalman filter is utilized to update the predictions of a gate estimate rather than raw PnP. IPPE-Square returns two candidate pose solutions, one of which is a flipped version of the frame, therefore raw updates to the gate estimate could result in an unwanted shift in values. By capturing the process noise vector P by incrementing each time a gate is not detected and decreasing when a gate is detected alongside a measurement noise vector R , the gain for an update becomes Equation 12.

$$K = P/(P + R). \quad (12)$$

This uncertainty and noise vector K is also used for an uncertainty-gated association algorithm, which rejects any detection that diverges from $N \cdot \sigma$ of the uncertainty, resulting in Equation 13. The algorithm also rejects any gate detected that is similar to the previous gate to prevent a relapse at the gate crossing. As shown in Fig. 3, the detections of a gate diverge as it approaches 2 meters of the gate; therefore, a gate detection is rejected whose depth is less than 2 meters and relies on dead reckoning. This is also matched in training such that the transition to vision inference is seamless. Finally, a stack was made to build a pose-estimation dataset in a YOLO format during data-collection rollouts that are automatically labelled from GT projections, which provided a way to gather data for vision model training without needing human annotation.

$$N \cdot \sqrt{P + R} \quad (13)$$

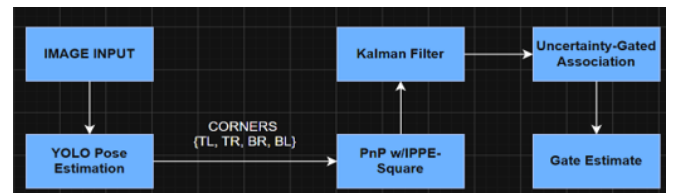


Fig. 2. Conceptual diagram of the perception pipeline during inference.

III. RESULTS

To evaluate the model, a few metrics are defined:

- Completion Rate: Fraction of runs that crossed gate index N , aggregated over 100 different course variations.
- Residual: $P_{vision} - P_{GT}$, every timestep that the target gate was detected in. P is defined as the gate position and is binned by true distance.
- X/Y and Y/Z trajectories to measure vertical and lateral movements during inference (color-coded with speed).
- X/Z spread of the position where the drone crossed the gate (green for cross, red for hits).

During evaluation, 100 rollouts were performed with 96 course generations unseen by the model during training, the maximum allocated gates were 14, and the maximum time allowed was 40.0s. Fig. 3 shows the completion rates of both ground-truth gate state and inference using vision. Vision tracks ground truth closely early (89% vs 96% at gate 1, 7-point gap) and diverges with depth (52% vs 80% by gate 13, 28-point gap) showcasing that error compounds faster under vision estimates. This not only demonstrates the policy’s generalization to unseen courses over long-horizon trajectories, but the intended goal of transferring from privileged ground truth to vision estimates. The compounding stems from the distance-dependent perception error that is characterized in the residual graph (Fig. 4). Most notably, the Y-axis has a mean error of -15 meters when detections are beyond 15 meters, a gate at 15m spans only 18px in the image, so small corner-localization errors translate into large depth errors, the source of the distance-dependent residual. Fig. 5 and Fig. 6 show a representative vision-estimated trajectory threading the full course, colored by speed. The path remains relatively centered through most gates, with only modest lateral and vertical corrections. Fig. 7 shows the corresponding gate crossings; the majority of successful passes occur near the center of the opening, indicating that the policy does not rely on extreme last-moment adjustments when the vision estimates remain usable. During the Anduril AI Grand Prix competition, the policy built was able to navigate through Virtual Qualifier 1 (shown in Fig. 8) and progressed partway through Virtual Qualifier 2, consistent with the 65% retention of privileged-state performance observed in the controlled evaluation.

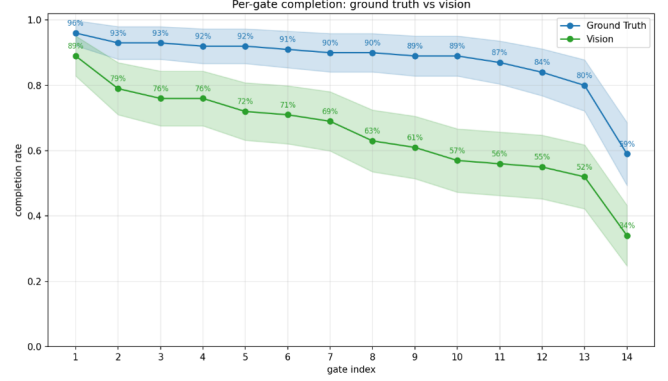


Fig. 3. Completion Rate % per-gate of ground truth (denoted in blue) and vision (green).

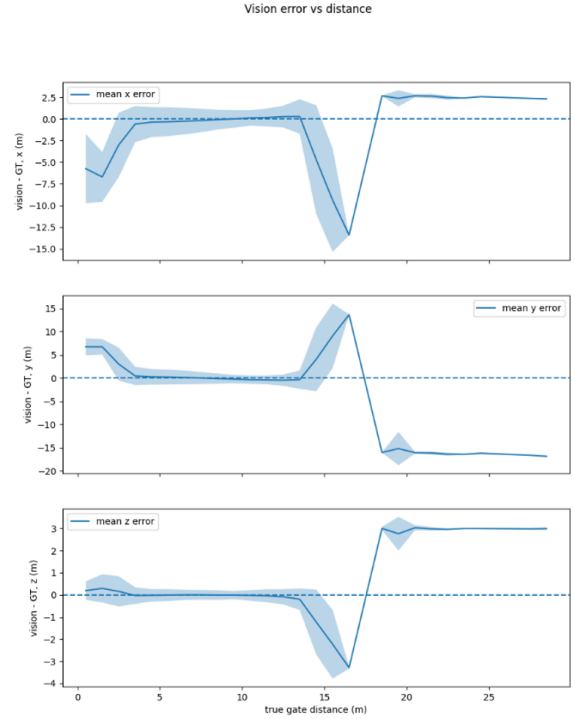


Fig. 4. Vision Residual aggregated over 100 rollouts.

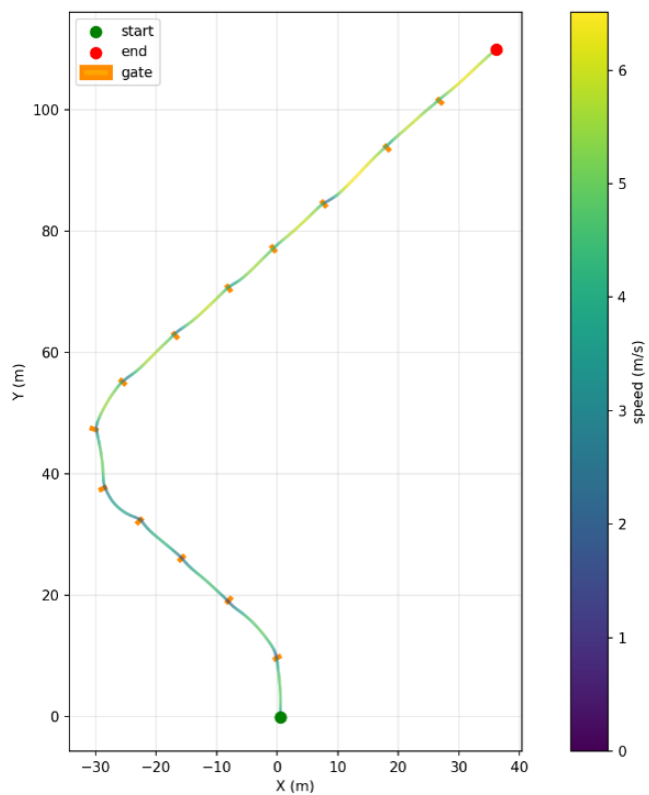


Fig. 5. Single trajectory on the X/Y axis using only vision inference.

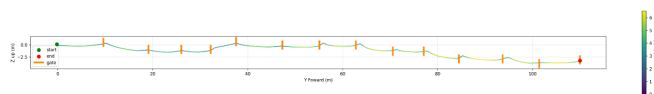


Fig. 6. Single trajectory on the Y/Z axis using only vision inference.

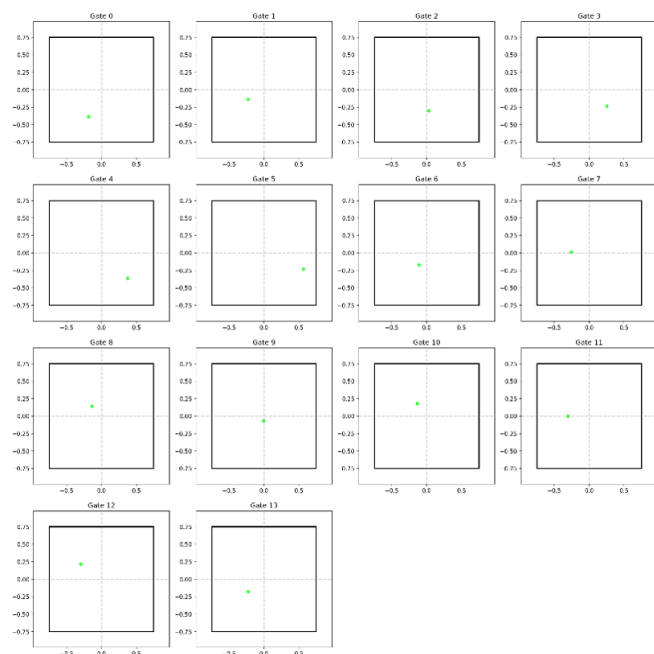


Fig. 7. Position of a quadrotor localized to each gate's pose after crossing during a single trajectory.

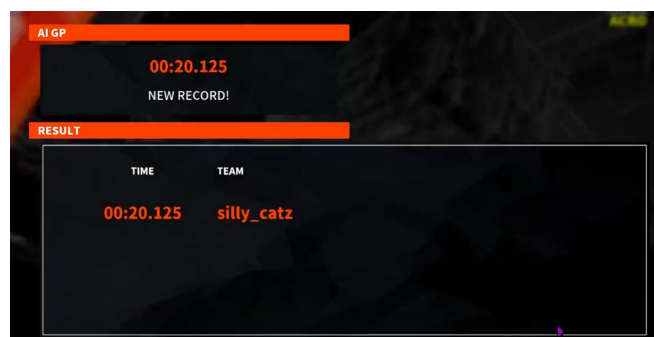


Fig. 8. Completion of Virtual Qualifier 1

REFERENCES

- [1] S. A. Bahnam et al., “MonoRace: Winning Champion-Level Drone Racing with Robust Monocular AI,” arXiv.org, 2026. <https://arxiv.org/abs/2601.15222>
- [2] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza, “Flightmare: A Flexible Quadrotor Simulator,” arXiv.org, 2020. <https://arxiv.org/abs/2009.00563>
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv.org, Aug. 28, 2017. <https://arxiv.org/abs/1707.06347>
- [4] R. Ferede, B. Till, E. Lucassen, De W. Christophe, and de C. G. C. H. E., “One Net to Rule Them All: Domain Randomization in Quadcopter Racing Across Different Platforms,” arXiv.org, 2025. <https://arxiv.org/abs/2504.21586>
- [5] G. Cano Lopes, M. Ferreira, A. da Silva Simões and E. Luna Colombini, "Intelligent Control of a Quadrotor with Proximal Policy Optimization Reinforcement Learning," pp. 503-508, 2018, doi: 10.1109/LARS/SBR/WRE.2018.00094.
- [6] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, “On the Continuity of Rotation Representations in Neural Networks,” arXiv.org, 2018. <https://arxiv.org/abs/1812.07035>
- [7] T. Collins and A. Bartoli, “Infinitesimal Plane-Based Pose Estimation,” International Journal of Computer Vision, vol. 109, no. 3, pp. 252–286, July 2014, doi: 10.1007/s11263-014-0725-5
- [8] Kaufmann, E., Bauersfeld, L., Loquercio, A. et al. Champion-level drone racing using deep reinforcement learning. Nature 620, 982–987 (2023). <https://doi.org/10.1038/s41586-023-06419-4>.